ELK Movies Platform

Rapport de synthèse

MD4 — Big Data · Avril 2026

Membres

BENBOUABDELLAH Melissa · AKOUDAD Elias · CONG Hoa · KANTE Ismaël Kennedy

Stack technique

Elasticsearch 8.13 · Logstash · Kibana · Python Flask

1. Contexte, objectifs et périmètre

1.1 Contexte

Ce projet s'inscrit dans le cadre du cours Big Data. L'objectif est de construire une plateforme d'analyse de films complète en s'appuyant sur la stack ELK (Elasticsearch, Logstash, Kibana), depuis l'ingestion brute des données jusqu'à la restitution visuelle et la mise à disposition d'un moteur de recherche interactif.

1.2 Objectifs réalisés

Objectif	Statut
Stack ELK déployée via Docker Compose	Réalisé
Ingestion brute dans movies_raw	Réalisé
Nettoyage et typage dans movies_clean	Réalisé
Mapping explicite + analyzer custom	Réalisé
12 requêtes DSL (dont 5 bool)	Réalisé
Dashboard Kibana (6–8 visualisations)	Réalisé
Mini moteur de recherche fonctionnel	Réalisé
Documentation complète (runbook, dictionnaire, cleaning)	Réalisé
GIF de démonstration (docs/demo.gif)	Réalisé

1.3 Périmètre non réalisé

La génération automatique de recommandations de films (champ recommandations) n'a pas été exploitée faute de temps. Le champ est stocké dans l'index **movies_clean** mais non indexé pour la recherche.

2. Architecture et environnement

2.1 Stack technique

La plateforme est entièrement conteneurisée via Docker Compose. Trois services principaux sont déployés :

Service	Image	Port	Rôle
elasticsearch	elasticsearch:8.13.0	9200	Moteur de stockage et de recherche
kibana	kibana:8.13.0	5601	Interface de visualisation
logstash	logstash:8.13.0	—	Pipeline d'ingestion et de nettoyage
Flask (local)	Python 3.x	5000	Mini moteur de recherche (hors Docker)

2.2 Flux de données

```
movies.csv  →  Logstash (parse CSV + clone)  →  movies_raw (données brutes)
                                  →  movies_clean (types convertis, dates parsées, listes normalisées)
                                       ↓              ↓
                                     Kibana         Flask API
```

2.3 Volumes Docker

- ./logstash/pipeline/logstash.conf monté en lecture seule dans le conteneur Logstash
- ./DATA monté en lecture seule pour l'accès au CSV
- ./elasticsearch/mapping.json appliqué via PUT avant le démarrage de Logstash

3. Données et nettoyage

3.1 Description du dataset

Le fichier movies.csv contient 20 colonnes couvrant les métadonnées TMDB : identifiant, titre, genres, langue originale, résumé, popularité, sociétés de production, date de sortie, budget, recettes, durée, statut, slogan, note moyenne, nombre de votes, crédits, mots-clés, chemins d'affiches et recommandations.

3.2 Anomalies détectées

Champ	Anomalie	Volume
id	Doublons dans le CSV	107 548 entrées dupliquées
title	Valeurs vides ou nulles	~4 films
overview	Valeurs nulles	~114 502 films (17,29 %)
genres	Valeurs nulles	~212 149 films (32,04 %)
release_date	Valeurs nulles ou mal formatées	~59 459 films (8,98 %)
tagline, keywords	Valeurs nulles très fréquentes	Non dénombrées
Champs numériques	Tous stockés en string par le CSV	7 champs concernés

3.3 Règles de nettoyage appliquées dans Logstash

Conversion de types — les champs numériques sont tous convertis depuis leur format string CSV :

Champ	Avant (CSV)	Après (ES)
id	"12345"	12345 (integer)
popularity	"84.5"	84.5 (float)
budget, revenue	"150000000"	150000000 (integer)
runtime	"148.0"	148.0 (float)
vote_average	"8.3"	8.3 (float)
vote_count	"24000"	24000 (integer)

Parsing de date — release_date est converti depuis le format yyyy-MM-dd vers un timestamp ISO 8601 compatible Elasticsearch. Les échecs de parsing sont taggés _date_parse_failure.

Normalisation des listes — genres et keywords sont stockés dans le CSV comme des chaînes séparées par des tirets. Logstash les découpe en tableaux :

```
Brut    : "Action-Science Fiction-Horror"
Nettoyé : ["Action", "Science Fiction", "Horror"]
```

Déduplication — le champ document_id est valorisé avec l'id TMDB. Elasticsearch ne crée donc qu'un seul document par film, même en cas de relance du pipeline.

Suppression des champs techniques — les champs internes Logstash (@version, host, path, message, event) sont retirés avant indexation dans movies_clean.

3.4 Mesure d'impact avant/après

Métrique	movies_raw (avant)	movies_clean (après)
Nombre de documents	769 631	662 083
Doublons	107 548	0
Type de release_date	string	date ISO 8601
Type de vote_average	string	float
Type de budget	string	integer
Format de genres	"Action-Drama"	["Action", "Drama"]
Champs techniques Logstash	présents	supprimés

La réduction de 13,97 % du volume de documents (107 548 doublons supprimés) garantit l'unicité des films dans l'index et améliore la qualité des agrégations et visualisations.

4. Modélisation Elasticsearch

4.1 Mapping de movies_clean

Le mapping est défini explicitement via PUT /movies_clean avant toute ingestion, évitant ainsi les conflits de type automatique d'Elasticsearch.

Champ	Type ES	Justification
title	text + .keyword	Recherche full-text + tri/agrégation exact
overview, tagline	text	Recherche full-text uniquement
genres, keywords	keyword	Agrégations Kibana, filtres exacts
original_language	keyword	Filtre exact par langue
status	keyword	Facette (Released, Post Production...)
release_date	date	Requêtes de plage temporelle
popularity, vote_average, runtime	float	Tri, scoring, agrégations numériques
budget, revenue, vote_count, id	integer / long	Calculs et agrégations
poster_path, recommendations	keyword (non indexé)	Stockage uniquement

4.2 Analyzer personnalisé

Deux analyzers customs sont définis : movies_text_analyzer (indexation) et movies_search_analyzer (recherche), partageant la même chaîne de traitement :

Étape	Filtre	Effet
1	html_strip	Supprime les balises HTML résiduelles
2	standard tokenizer	Découpe le texte en tokens
3	lowercase	Normalise la casse (Spider-Man → spider-man)
4	asciifolding	Convertit les accents (é → e, ü → u)
5	english_stop	Supprime les mots vides (the, a, is, of...)
6	english_stemmer	Réduit les mots à leur racine (running → run)

Exemple : entrée "The Amazing Spider-Man returns" → tokens produits : ["amaz", "spider", "man", "return"]

Ce pipeline améliore le recall (retrouve plus de résultats pertinents) tout en conservant la précision grâce au stemming et à la suppression des stop words.

5. Requêtes et analyses

Douze requêtes Elasticsearch ont été rédigées et commentées dans le notebook docs/queries_dsl.ipynb, couvrant des cas d'usage métier variés.

5.1 Requêtes simples (7 requêtes)

ID	Type	Objectif métier
R-01	match	Recherche d'un film par titre avec l'analyzer custom
R-02	range (release_date)	Films sortis entre deux années
R-03	term (original_language)	Films en français uniquement
R-04	range (vote_average)	Films très bien notés (≥ 8.0)
R-05	multi_match	Recherche sur titre ET overview simultanément
R-06	wildcard	Films dont le titre commence par un préfixe
R-07	fuzzy	Recherche tolérante aux fautes de frappe

5.2 Requêtes bool (5 requêtes)

Bool 1 — Films d'action anglophones populaires récents

```
{ "bool": { "must": [{ "term": { "genres": "Action" } }],
        "filter": [{ "term": { "original_language": "en" } },
                { "range": { "release_date": { "gte": "2015-01-01" } } },
                { "range": { "popularity": { "gte": 50 } } }] } }
```

Bool 2 — Films bien notés, hors films d'horreur

```
{ "bool": { "must": [{ "range": { "vote_average": { "gte": 7.5 } } }],
        "must_not": [{ "term": { "genres": "Horror" } }],
        "filter": [{ "range": { "vote_count": { "gte": 500 } } }] } }
```

Bool 3 — Recherche full-text avec boost sur le titre

```
{ "bool": { "should": [{ "match": { "title": { "query": "space", "boost": 3 } } },
                { "match": { "overview": { "query": "space", "boost": 1 } } }],
        "minimum_should_match": 1 } }
```

Bool 4 — Blockbusters à gros budget avec bonne note

```
{ "bool": { "filter": [{ "range": { "budget": { "gte": 100000000 } } },
                { "range": { "vote_average": { "gte": 7.0 } } },
                { "range": { "vote_count": { "gte": 1000 } } }] } }
```

Bool 5 — Films par genre et décennie (ex. : Sci-Fi des années 2000)

```
{ "bool": { "must": [{ "term": { "genres": "Science Fiction" } }],
        "filter": [{ "range": { "release_date": { "gte": "2000-01-01", "lte": "2009-12-31" } } }] } }
```

6. Dashboard Kibana et lecture métier

Le dashboard Movies Analysis regroupe 7 visualisations construites avec Kibana Lens :

7. Gestion de projet et collaboration

7.1 Organisation Gitflow

Le dépôt respecte le Gitflow imposé : branche main (stable), dev (intégration) et branches feature/<id>-<slug> par fonctionnalité. Les pushs directs sur main et dev sont interdits — toute intégration passe par une Pull Request avec au moins 1 reviewer.

7.2 Traçabilité des Pull Requests

PR	Feature	Auteur	Reviewer
#1	US-01 — Bootstrap & Ingestion	KANTE Ismaël Kennedy	BENBOUABDELLAH Melissa
#2	US-06 — Documentation	BENBOUABDELLAH Melissa	CONG Hoa
#4	US-03 — Requêtes DSL	KANTE Ismaël Kennedy	AKOUDAD Elias
#6	US-02 — Mapping & Analyzer	CONG Hoa	KANTE Ismaël Kennedy
#7	US-04 — Dashboard Kibana	AKOUDAD Elias	BENBOUABDELLAH Melissa
#8	US-02 — Corrections post-review	CONG Hoa	AKOUDAD Elias
#9	US-05 — Mini moteur de recherche	CONG Hoa	KANTE Ismaël Kennedy

Chaque membre a ouvert au moins une PR et reviewé au moins une PR, conformément aux exigences du projet.

7.3 Répartition des features

Membre	Features
BENBOUABDELLAH Melissa	US-06 — Documentation & Démo (runbook, GIF, rapport)
AKOUDAD Elias	US-03 — Requêtes DSL · US-04 — Dashboard Kibana
CONG Hoa	US-02 — Mapping & Analyzer · US-05 — Mini moteur de recherche
KANTE Ismaël Kennedy	US-01 — Bootstrap & Ingestion · US-03 — Requêtes DSL

7.4 Planning poker

Session réalisée en début de projet sur 6 user stories, estimation en échelle Fibonacci (1, 2, 3, 5, 8, 13).

ID	User Story (résumé)	Votes	Final	Owner
US-01	Bootstrap, Ingestion & Nettoyage ELK + CSV	3,5,5,3	5	KANTE
US-02	Mapping & Analyzer custom sur movies_clean	2,3,3,3	3	CONG
US-03	12 requêtes Elasticsearch dont 5 bool	3,5,5,3	5	KANTE / AKOUDAD
US-04	Dashboard Kibana 6–8 visualisations	3,3,5,3	3	AKOUDAD
US-05	Mini moteur de recherche Flask + filtres	5,8,5,5	5	CONG
US-06	Documentation, GIF et rapport 5 pages	3,5,3,5	5	BENBOUABDELLAH

8. Bilan, limites et améliorations

8.1 Points forts

- **Pipeline Logstash robuste avec déduplication automatique par document_id**
- Analyzer custom adapté au corpus anglophone (stemming + ASCII folding + stop words)
- Moteur de recherche avec combinaison de 3 stratégies (full-text, préfixe, wildcard) et scoring boosté par la popularité
- Projet entièrement reproductible via docker compose up -d (voir runbook)
- Documentation complète : dictionnaire de données, data_cleaning.md, runbook, planning poker

8.2 Limites rencontrées

Limite	Impact
32 % des films sans genre	Agrégations par genre incomplètes
17 % des films sans overview	Recherche full-text dégradée pour ces films
Budget/revenue à 0 pour beaucoup de films	Analyses financières peu fiables
Logstash lent sur 769 631 lignes	Temps d'ingestion initial long (~15–20 min)
Champ recommendations non exploité	Pas de moteur de recommandation collaboratif

8.3 Améliorations possibles

- Ingestion incrémentale — ne réingérer que les films mis à jour via l'API TMDB plutôt que tout le CSV.
- Synonymes dans l'analyzer — ajouter un filtre pour que 'movie' = 'film' = 'cinema'.
- Pagination curseur (search_after) — plus performante que from/size pour les grands index.
- Affichage des affiches — utiliser poster_path avec l'API TMDB pour afficher les visuels dans le moteur.
- Tests automatisés — ajouter des tests sur les requêtes DSL pour détecter les régressions de mapping.